

OSSP (Operating Systems And Systems Programming) - 25CS2104E

1. Develop a C program that demonstrates how a Linux operating system executes a command entered by a user that 1. Accept a Linux command as input. 2. Create a child process using fork(). 3. Execute the command in the child process using an appropriate exec() system call. 4. Allow the parent process to wait for the child using wait (). 5. Display the Process ID (PID) of both parent and child processes.

Using Linux terminal commands (uname, lscpu, lsblk, ps, top), investigate the relationship between hardware resources and operating system services. Prepare a report explaining how the OS abstracts CPU, memory, storage, and I/O devices.

CODE:

```
#include <stdio.h>#include <stdlib.h>#include <unistd.h>#include <sys/wait.h>int main() { char command[100]; printf("Enter command:"); scanf("%99s", command); pid_t pid = fork(); if (pid < 0) { perror("fork"); return 1; } if (pid == 0) { printf("Child PID: %d\n", getpid()); execlp(command, command, (char *)NULL); perror("exec"); exit(1); } else { printf("Parent PID: %d\n", getpid()); printf("Waiting for child...\n"); wait(NULL); printf("Child completed.\n"); } return 0;}
```

OUTPUT:

```
leeshayuktha@Eeshas-MacBook-Air OSLab % nano relate
leeshayuktha@Eeshas-MacBook-Air OSLab % nano relate.c
leeshayuktha@Eeshas-MacBook-Air OSLab % gcc relate.c -o rel
leeshayuktha@Eeshas-MacBook-Air OSLab % ./rel
Enter a command to execute (e.g., ls, date, pwd): ls
Parent Process ID (PID): 37638
Parent waiting for child process...
Child Process ID (PID): 37640
ba          copycont.c      fork.c       pa          prac1w2.c.save  prun.c       relate
baexec      file                  hello       pa.c        printname.c     readUsername  relate.c
baexec.c    file1.txt             hello.c     pid         pro             readUsername.c SampleProgram.c
cc          filecopy.c            hi          pid.c       process.c       rel          un
childcomp.c fork                  name        prac1w2.c   prun           rela.c
Child process completed.
leeshayuktha@Eeshas-MacBook-Air OSLab % lscpu
zsh: command not found: lscpu
leeshayuktha@Eeshas-MacBook-Air OSLab % ps
  PID TTY          TIME CMD
 37478 ttys000    0:00.08 -zsh
leeshayuktha@Eeshas-MacBook-Air OSLab % uname
Darwin
leeshayuktha@Eeshas-MacBook-Air OSLab %
```

1.a C program that uses the system calls `open()`, `read()`, `write()`, and `close()` to copy the contents of one file to another. Explain how control transitions between user space and kernel space during execution.

Use the `strace` utility to trace all system calls generated by the command `cat sample.txt`. Analyze the sequence of system calls and identify the kernel services involved.

CODE:

```
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <unistd.h>

#define BUFFER_SIZE 1024

int main() {
    int src_fd, dest_fd;
    ssize_t bytes_read, bytes_written;
    char buffer[BUFFER_SIZE];

    // Open source file for reading
    src_fd = open("source.txt", O_RDONLY);
    if (src_fd == -1) {
        perror("Error opening source file");
        return 1;
    }

    // Open destination file for writing (Create if not existing, truncate if
    exists)
    dest_fd = open("destination.txt", O_WRONLY | O_CREAT | O_TRUNC,
0644);
    if (dest_fd == -1) {
        perror("Error opening destination file");
        close(src_fd);
        return 1;
    }

    // Copy file contents
    while ((bytes_read = read(src_fd, buffer, BUFFER_SIZE)) > 0) {
        bytes_written = write(dest_fd, buffer, bytes_read);
        if (bytes_written != bytes_read) {
            perror("Error writing to destination file");
            close(src_fd);
            close(dest_fd);
            return 1;
        }
    }
}
```

```

}

if (bytes_read == -1) {
    perror("Error reading source file");
} else {
    printf("File copied successfully.\n");
}

close(src_fd);
close(dest_fd);
return 0;
}

```

OUTPUT:

```

eeshayuktha@Eeshas-MacBook-Air OSLab % ./f
Error opening source file: No such file or directory
eeshayuktha@Eeshas-MacBook-Air OSLab %

```

1. Develop a C program using `fork()` that creates a parent and child process. Display the Process ID (PID), Parent Process ID (PPID), and process states at different stages of execution.

Design an experiment to observe process state transitions (Ready, Running, Waiting, Terminated) using Linux monitoring tools such as `ps`, `top`, and `/proc`. Document the observations.

CODE:

```

#include <stdio.h>

#include <stdlib.h>

#include <unistd.h>

#include <sys/types.h>

#include <sys/wait.h>

int main() {

    pid_t pid = fork();

    if (pid < 0) {

        perror("Fork failed");
    }
}

```

```

        return 1;
    } else if (pid == 0) {
        // Child process
        printf("Child Process (PID: %d), Parent PID: %d\n", getpid(), getppid());
        sleep(2); // Sleep to demonstrate orphan process if parent terminates
        printf("Child Process (PID: %d) after sleep, Parent PID: %d\n", getpid(),
getppid());
        exit(0);
    } else {
        // Parent process creates Zombie scenario by not calling wait()
immediately
        printf("Parent Process (PID: %d). Child PID is %d\n", getpid(), pid);
        printf("Parent sleeping... Check 'ps' in another terminal to see zombie
child.\n");
        sleep(5);

        // Cleanup zombie
        wait(NULL);
        printf("Parent collected child status, zombie state resolved.\n");
    }

    return 0;
}

```

OUTPUT:

```

[eeshayuktha@Eeshas-MacBook-Air OSLab % gcc zomb.c -o zombie
[eeshayuktha@Eeshas-MacBook-Air OSLab % ./zombie
Parent Process (PID: 37885). Child PID is 37886
Parent sleeping... Check 'ps' in another terminal to see zombie child.
Child Process (PID: 37886), Parent PID: 37885
Child Process (PID: 37886) after sleep, Parent PID: 37885
Parent collected child status, zombie state resolved.
eeshayuktha@Eeshas-MacBook-Air OSLab % █

```

1. Write a C program where a parent process creates multiple child processes and synchronizes their completion using wait() and waitpid(). Compare the behavior of both functions.

Create a scenario where a child process becomes a zombie process. Investigate the process table and then modify the program to eliminate zombie processes using proper synchronization techniques.

CODE:

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

int main() {
    pid_t pids[3];

    for (int i = 0; i < 3; i++) {
        pids[i] = fork();
        if (pids[i] < 0) {
            perror("Fork failed");
            return 1;
        } else if (pids[i] == 0) {
            // Child process
            printf("Child %d running (PID: %d)\n", i + 1, getpid());
            sleep(i + 1); // Simulate distinct tasks
            printf("Child %d finished (PID: %d)\n", i + 1, getpid());
            exit(0);
        }
    }

    // Parent waits for all child processes sequentially
    for (int i = 0; i < 3; i++) {
        int status;
```

```

        pid_t completed_pid = waitpid(pids[i], &status, 0);
        printf("Parent synchronized with Child PID %d\n", completed_pid);
    }

    printf("All children have finished executing.\n");
    return 0;
}

```

OUTPUT:

```

[eeshayuktha@Eeshas-MacBook-Air OSLab % nano sync.c
[eeshayuktha@Eeshas-MacBook-Air OSLab % gcc sync.c -o sync
[eeshayuktha@Eeshas-MacBook-Air OSLab % ./sync
Child 1 running (PID: 37932)
Child 2 running (PID: 37933)
Child 3 running (PID: 37934)
Child 1 finished (PID: 37932)
Parent synchronized with Child PID 37932
Child 2 finished (PID: 37933)
Parent synchronized with Child PID 37933
Child 3 finished (PID: 37934)
Parent synchronized with Child PID 37934
All children have finished executing.
eeshayuktha@Eeshas-MacBook-Air OSLab % █

```

1. Implement a producer-consumer communication system using anonymous pipes where the parent process generates data and the child process consumes it. Measure the communication efficiency.

Develop a program that executes the equivalent of the shell command: `ls -l | grep ".c"` using `fork()`, `pipe()`, `dup2()`, and `exec()` system calls.

CODE:

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>

```

```

int main() {

```

```
int fd[2];
char message[] = "Hello from Child via Anonymous Pipe!";
char buffer[100];

if (pipe(fd) == -1) {
    perror("Pipe failed");
    return 1;
}

pid_t pid = fork();

if (pid < 0) {
    perror("Fork failed");
    return 1;
} else if (pid == 0) {
    // Child Process writes to pipe
    close(fd[0]); // Close unused read end
    write(fd[1], message, strlen(message) + 1);
    close(fd[1]);
    exit(0);
} else {
    // Parent Process reads from pipe
    close(fd[1]); // Close unused write end
    read(fd[0], buffer, sizeof(buffer));
    printf("Parent received message: %s\n", buffer);
    close(fd[0]);
}

return 0;
}
```

OUTPUT:

```
[eeshayuktha@Eeshas-MacBook-Air OSLab % nano ipc.c
[eeshayuktha@Eeshas-MacBook-Air OSLab % nano ipc.c
[eeshayuktha@Eeshas-MacBook-Air OSLab % gcc ipc.c -o ipc
[eeshayuktha@Eeshas-MacBook-Air OSLab % ./ipc
Parent received message: Hello from Child via Anonymous Pipe!
eeshayuktha@Eeshas-MacBook-Air OSLab % █
```

1. Create a client-server application using Named Pipes (FIFOs). The client sends messages and the server processes and responds to them. Analyze FIFO behavior when multiple clients communicate simultaneously.

Create a POSIX signal handling program that captures SIGINT, SIGTERM, and SIGUSR1. Demonstrate asynchronous event handling and explain the role of signal handlers.

CODE:

```
#include <stdio.h>
#include <stdlib.h>
#include <signal.h>
#include <unistd.h>

void signal_handler(int sig) {
    if (sig == SIGINT) {
        printf("\nCaught SIGINT (Ctrl+C). Exiting gracefully...\n");
        exit(0);
    } else if (sig == SIGTERM) {
        printf("\nCaught SIGTERM.\n");
    } else if (sig == SIGUSR1) {
        printf("\nCaught SIGUSR1 signal.\n");
    }
}

int main() {
    struct sigaction sa;
    sa.sa_handler = signal_handler;
    sigemptyset(&sa.sa_mask);
    sa.sa_flags = 0;

    sigaction(SIGINT, &sa, NULL);
    sigaction(SIGTERM, &sa, NULL);
```

```

sigaction(SIGUSR1, &sa, NULL);

printf("Process PID: %d. Waiting for signals...\n", getpid());
while (1) {
    sleep(1);
}

return 0;
}

```

OUTPUT:

```

[eeshayuktha@Eeshas-MacBook-Air OSLab % nano posix.c
[eeshayuktha@Eeshas-MacBook-Air OSLab % gcc posix.c -o po
[eeshayuktha@Eeshas-MacBook-Air OSLab % ./po
Process PID: 38083. Waiting for signals...

```

1. Write a program that prints the addresses of code, global, static, heap, and stack variables. Analyze the Linux process address space layout and explain each memory segment.

Make use of the `/proc//maps` file and memory analysis tools to study the memory organization of a running process. Prepare a report explaining virtual memory mappings.

CODE:

```

#include <stdio.h>
#include <stdlib.h>

int global_var = 10;        // Initialized Data Segment
int uninit_global_var;      // BSS Segment

void sample_function() {}   // Code/Text Segment

int main() {
    static int static_var = 20; // Data Segment
    int stack_var = 30;         // Stack Segment
    int *heap_var = (int *)malloc(sizeof(int)); // Heap Segment

```

```

printf("Code/Text Segment Address: %p\n", (void *)sample_function);
printf("Global Var Address (Data): %p\n", (void *)&global_var);
printf("Static Var Address (Data): %p\n", (void *)&static_var);
printf("BSS Var Address:          %p\n", (void *)&uninit_global_var);
printf("Heap Var Address:         %p\n", (void *)heap_var);
printf("Stack Var Address:        %p\n", (void *)&stack_var);

free(heap_var);
return 0;
}

```

OUTPUT:

1. Develop a program using malloc(), calloc(), realloc(), and free(). Monitor memory allocation behavior and identify memory leaks using Valgrind.

```

[eeshayuktha@Eeshas-MacBook-Air ~ % cd ~/Documents/OSLab
[eeshayuktha@Eeshas-MacBook-Air OSLab % nano memley.c
[eeshayuktha@Eeshas-MacBook-Air OSLab % gcc memley.c -o mem
[eeshayuktha@Eeshas-MacBook-Air OSLab % ./mem
Code/Text Segment Address: 0x1009f8548
Global Var Address (Data): 0x100a00000
Static Var Address (Data): 0x100a00004
BSS Var Address:          0x100a00008
Heap Var Address:         0x101181970
Stack Var Address:        0x16f407298
eeshayuktha@Eeshas-MacBook-Air OSLab % █

```

Develop a program that demonstrates Copy-on-Write behavior after a fork(). Analyze memory usage before and after modifying data in the child process.

CODE:

```

#include <stdio.h>
#include <stdlib.h>

```

```

#include <string.h>
#include <fcntl.h>
#include <unistd.h>
#include <sys/mman.h>

#define SHM_SIZE 1024

int main() {
    // Map anonymous shared memory
    char *shared_mem = mmap(NULL, SHM_SIZE, PROT_READ | PROT_WRITE,
MAP_SHARED | MAP_ANONYMOUS, -1, 0);

    if (shared_mem == MAP_FAILED) {
        perror("mmap failed");
        return 1;
    }

    pid_t pid = fork();

    if (pid < 0) {
        perror("Fork failed");
        return 1;
    } else if (pid == 0) {
        // Child process writes data
        strcpy(shared_mem, "Hello World from Shared Memory!");
        msync(shared_mem, SHM_SIZE, MS_SYNC);
        exit(0);
    } else {
        // Parent process waits for child and reads data
        wait(NULL);
        printf("Parent read from shared memory: %s\n", shared_mem);
        munmap(shared_mem, SHM_SIZE);
    }
}

```

```
    return 0;
}
```

OUTPUT:

```
[eeshayuktha@Eeshas-MacBook-Air OSLab % gcc sharedmem.c -o share
[eeshayuktha@Eeshas-MacBook-Air OSLab % ./share
Parent read from shared memory: Hello World from Shared Memory!
eeshayuktha@Eeshas-MacBook-Air OSLab % █
```

1. Implement a file copy utility using low-level Linux file I/O system calls (open, read, write, lseek, close). Compare its performance with standard library functions (fopen, fread, fwrite).

Develop a program that redirects standard input and output using file descriptors and dup2(). Demonstrate how shells implement I/O redirection.

CODE:

```
#include <stdio.h>

#include <fcntl.h>

#include <unistd.h>

int main()
{
    int source;
    int destination;

    char buffer[1024];
    ssize_t bytes;

    source = open(
        "source.txt",
        O_RDONLY
    );
```

```
if (source < 0)
{
    perror("source.txt");
    return 1;
}

destination = open(
    "destination.txt",
    O_WRONLY | O_CREAT | O_TRUNC,
    0644
);

if (destination < 0)
{
    perror("destination.txt");
    close(source);
    return 1;
}

while (
    (bytes = read(
        source,
        buffer,
        sizeof(buffer)
    )) > 0
)
{
    write(
        destination,
```

```
        buffer,  
        bytes  
    );  
}
```

```
lseek(  
    source,  
    0,  
    SEEK_SET  
);
```

```
close(source);  
close(destination);
```

```
printf("File copied successfully.\n");
```

```
    return 0;  
}
```

CODE2:

```
#include <stdio.h>  
#include <fcntl.h>  
#include <unistd.h>
```

```
int main()  
{  
    int input;  
    int output;
```

```
    input = open(
```

```
    "input.txt",  
    O_RDONLY  
);
```

```
output = open(  
    "output.txt",  
    O_WRONLY | O_CREAT | O_TRUNC,  
    0644  
);
```

```
if (input < 0 || output < 0)  
{  
    perror("open");  
    return 1;  
}
```

```
dup2(  
    input,  
    STDIN_FILENO  
);
```

```
dup2(  
    output,  
    STDOUT_FILENO  
);
```

```
close(input);  
close(output);
```

```
printf(  

```

```
    "This output is redirected to output.txt\n"
);

return 0;
}
```

OUTPUT :

```
[eeshayuktha@Eeshas-MacBook-Air OSLab % nano lseek.c
[eeshayuktha@Eeshas-MacBook-Air OSLab % nano source.txt
[eeshayuktha@Eeshas-MacBook-Air OSLab % nano desitination.txt
[eeshayuktha@Eeshas-MacBook-Air OSLab % gcc lseek.c -o seek
[eeshayuktha@Eeshas-MacBook-Air OSLab % ./seek
File copied successfully.
eeshayuktha@Eeshas-MacBook-Air OSLab %
```

1. Investigate inode structures using commands such as `ls -li`, `stat`, and `find`. Create hard links and symbolic links and explain their impact on inode allocation.

Design a program using `mmap()` to perform file reading and writing. Compare memory-mapped I/O with traditional read/write operations in terms of efficiency and implementation complexity.

CODE:

```
#include <stdio.h>
#include <unistd.h>

int main()
{
    FILE *fp;

    fp = fopen(
```

```
    "original.txt",
    "w"
);

if (fp == NULL)
{
    perror("original.txt");
    return 1;
}

fprintf(
    fp,
    "This is the original file.\n"
);

fclose(fp);

if (link(
    "original.txt",
    "hardlink.txt"
) == -1)
{
    perror("hard link");
}

if (symlink(
    "original.txt",
    "softlink.txt"
) == -1)
{
    perror("symbolic link");
}
```

```
printf("Links created successfully.\n");
```

```
return 0;
```

```
}
```

CODE2:

```
#include <stdio.h>
```

```
#include <fcntl.h>
```

```
#include <unistd.h>
```

```
#include <sys/mman.h>
```

```
#include <string.h>
```

```
int main()
```

```
{
```

```
    int fd;
```

```
    const char *text =
```

```
        "Hello using mmap!";
```

```
    size_t size =
```

```
        strlen(text) + 1;
```

```
    fd = open(
```

```
        "mmapfile.txt",
```

```
        O_RDWR | O_CREAT,
```

```
        0644
```

```
    );
```

```
    if (fd < 0)
```

```
    {
```

```
        perror("open");
```

```
        return 1;
```

```
    }
```

```
ftruncate(  
    fd,  
    size  
);
```

```
char *data = mmap(  
    NULL,  
    size,  
    PROT_READ | PROT_WRITE,  
    MAP_SHARED,  
    fd,  
    0  
);
```

```
if (data == MAP_FAILED)  
{  
    perror("mmap");  
    close(fd);  
    return 1;  
}
```

```
strcpy(  
    data,  
    text  
);
```

```
printf(  
    "File content: %s\n",  
    data  
);
```

```
msync(  
    data,
```

```

        size,
        MS_SYNC
    );

    munmap(
        data,
        size
    );

    close(fd);

    return 0;
}

```

OUTPUT1/2:

```

[eeshayuktha@Eeshas-MacBook-Air OSLab % nano hardlink.c
[eeshayuktha@Eeshas-MacBook-Air OSLab % gcc hardlink.c -o hl
[eeshayuktha@Eeshas-MacBook-Air OSLab % ./hl
Links created successfully.
eeshayuktha@Eeshas-MacBook-Air OSLab % █

```

```

[eeshayuktha@Eeshas-MacBook-Air OSLab % nano mmap.c
[eeshayuktha@Eeshas-MacBook-Air OSLab % gcc mmap.c -o map
[eeshayuktha@Eeshas-MacBook-Air OSLab % ./map
File content: Hello using mmap!
eeshayuktha@Eeshas-MacBook-Air OSLab % █

```

1. Develop a multithreaded counter application using POSIX threads (pthread_create, pthread_join). Demonstrate race conditions by allowing multiple threads to update a shared variable concurrently.

Modify the previous program using mutex locks (pthread_mutex_lock, pthread_mutex_unlock) and compare the outputs with the race-condition version. Analyze the impact of synchronization.

CODE:

```

#include <stdio.h>

#include <pthread.h>

```

```
int counter = 0;
```

```
void *increment(void *arg)
```

```
{  
    for (int i = 0; i < 100000; i++)  
    {  
        counter++;  
    }  
  
    return NULL;  
}
```

```
int main()
```

```
{  
    pthread_t thread1;  
    pthread_t thread2;
```

```
    pthread_create(  
        &thread1,  
        NULL,  
        increment,  
        NULL  
    );
```

```
    pthread_create(  
        &thread2,  
        NULL,  
        increment,  
        NULL  
    );
```

```

pthread_join(
    thread1,
    NULL
);

pthread_join(
    thread2,
    NULL
);

printf(
    "Final counter value: %d\n",
    counter
);

return 0;
}

```

CODE2:

```

#include <stdio.h>
#include <pthread.h>

int counter = 0;

pthread_mutex_t lock;

void *increment(void *arg)
{
    for (int i = 0; i < 100000; i++)

```

```
{  
    pthread_mutex_lock(&lock);  
  
    counter++;  
  
    pthread_mutex_unlock(&lock);  
}  
  
return NULL;  
}
```

```
int main()  
{  
    pthread_t thread1;  
    pthread_t thread2;  
  
    pthread_mutex_init(  
        &lock,  
        NULL  
    );  
  
    pthread_create(  
        &thread1,  
        NULL,  
        increment,  
        NULL  
    );  
  
    pthread_create(  
        &thread2,
```

```
        NULL,  
        increment,  
        NULL  
    );  
  
    pthread_join(  
        thread1,  
        NULL  
    );  
  
    pthread_join(  
        thread2,  
        NULL  
    );  
  
    printf(  
        "Final counter value: %d\n",  
        counter  
    );  
  
    pthread_mutex_destroy(  
        &lock  
    );  
  
    return 0;  
}
```

OUTPUT1/2:

```
[eeshayuktha@Eeshas-MacBook-Air OSLab % nano racecond.c
[eeshayuktha@Eeshas-MacBook-Air OSLab % gcc racecond.c -o rc
[eeshayuktha@Eeshas-MacBook-Air OSLab % ./rc
Final counter value: 103843
eeshayuktha@Eeshas-MacBook-Air OSLab % █

[eeshayuktha@Eeshas-MacBook-Air OSLab % nano mutex.c
[eeshayuktha@Eeshas-MacBook-Air OSLab % gcc mutex.c -o mutex
[eeshayuktha@Eeshas-MacBook-Air OSLab % ./mutex
Final counter value: 200000
eeshayuktha@Eeshas-MacBook-Air OSLab % █
```

1. Implement the Producer-Consumer problem using counting semaphores and POSIX threads. Evaluate synchronization correctness and system throughput under different buffer sizes.

Design a deadlock scenario involving multiple threads competing for shared resources. Demonstrate deadlock occurrence, identify the four necessary conditions for deadlock, and implement a deadlock prevention strategy using resource ordering.

CODE:

```
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

pthread_mutex_t resource1 =
    PTHREAD_MUTEX_INITIALIZER;

pthread_mutex_t resource2 =
    PTHREAD_MUTEX_INITIALIZER;

void *thread1(void *arg)
{
```

```
pthread_mutex_lock(
    &resource1
);

printf(
    "Thread 1 locked Resource 1\n"
);

sleep(1);

pthread_mutex_lock(
    &resource2
);

printf(
    "Thread 1 locked Resource 2\n"
);

pthread_mutex_unlock(
    &resource2
);

pthread_mutex_unlock(
    &resource1
);

return NULL;
}

void *thread2(void *arg)
{
```

```
/*  
    Resource ordering:  
    Always lock Resource 1 first,  
    then Resource 2.  
*/  
  
pthread_mutex_lock(  
    &resource1  
);  
  
printf(  
    "Thread 2 locked Resource 1\n"  
);  
  
sleep(1);  
  
pthread_mutex_lock(  
    &resource2  
);  
  
printf(  
    "Thread 2 locked Resource 2\n"  
);  
  
pthread_mutex_unlock(  
    &resource2  
);  
  
pthread_mutex_unlock(  
    &resource1  
);
```

```
        return NULL;
    }

int main()
{
    pthread_t t1;
    pthread_t t2;

    pthread_create(
        &t1,
        NULL,
        thread1,
        NULL
    );

    pthread_create(
        &t2,
        NULL,
        thread2,
        NULL
    );

    pthread_join(
        t1,
        NULL
    );

    pthread_join(
        t2,
        NULL
    );
}
```

```
);

printf(
    "No deadlock occurred because resources were locked in the same
order.\n"
);

return 0;
}
```

OUTPUT:

```
eeshayuktha@Eeshas-MacBook-Air OSLab % nano deadprev.c
eeshayuktha@Eeshas-MacBook-Air OSLab % gcc deadprev.c -o dead
eeshayuktha@Eeshas-MacBook-Air OSLab % ./dead
Thread 1 locked Resource 1
Thread 1 locked Resource 2
Thread 2 locked Resource 1
Thread 2 locked Resource 2
No deadlock occurred because resources were locked in the same order.
eeshayuktha@Eeshas-MacBook-Air OSLab %
```